Vulnerability and Protocol Association of Device Firmware in Power Grid

Jiye Wang
China Electric Power Research Institute
Beijing, China
wangjiye@epri.sgcc.com.cn

Yuyan Sun
Institute of Information Engineering
Chinese Academy of Sciences
Beijing, China
sunyuyan@iie.ac.cn

Siwei Miao
China Electric Power Research Institute
Beijing, China
miaosiwei@epri.sgcc.com.cn

Zhiqiang Shi
Institute of Information Engineering
Chinese Academy of Sciences
Beijing, China
sunyuyan@iie.ac.cn

Limin Sun
Institute of Information Engineering
Chinese Academy of Sciences
Beijing, China
sunyuyan@iie.ac.cn

***Abstract*—The intelligent power grid is composed of a large number of industrial control equipment, and most of the industrial control equipment has security holes, which are vulnerable to malicious attacks and affect the normal operation of the power grid. By analyzing the security vulnerability of the firmware of industrial control equipment, the vulnerability can be detected in advance and the power grid's ability to resist attack can be improved. In this paper, a kind of industrial control device firmware protocol vulnerabilities associated technology, through the technology of information extraction from the mass grid device firmware device attributes and extract the industrial control system, the characteristics of the construction of industrial control system device firmware and published vulnerability information correlation, faster in the industrial control equipment safety inspection found vulnerabilities.**

Keywords— vulnerability association, firmware preprocessing, features extraction

I. INTRODUCTION

Embedded devices, such as PLCs, RTUs, HMIs, engineer stations, operator stations, are widely used in the industrial control systems of smart grid. Embedded devices brings more security risks while making the power grid more networked, intelligent, and multi-functional. Embedded networkable devices are ubiquitous and widely deployed in privacy-sensitive and secure areas, which require these devices to be secure and reliable. However, unfortunately, research shows that most embedded devices have a large number of security risks and vulnerability. For example, there are vulnerabilities such as command injection and hard coding in a large number of embedded devices. At the same time, the firmware of related embedded systems still has backdoors implanted by vendors. On the other hand, the industrial control systems in the smart grid are important part of the country's key infrastructure, and its safety concerns the national economy, the people's livelihood, and the country's strategic security. The security of these devices largely determines the security of the power industrial control system. Once it is attacked, it will lead to power equipment failure and threaten the normal operation and stability of the smart grid, resulting in disastrous consequences. In 2016, the Ukrainian power company's network system was hacked, in which the firmware[1] of the power grid equipment was modified, causing massive power outages and thousands of families to spend in the dark.

II. RELATED WORKS

A. Firmware preprocessing

Because the firmware contains the file system and communication environment necessary to maintain the normal operation of the device, the current security research and analysis of embedded devices focus on the firmware. For the structure and design of firmware have different manufacturers, categories, and models. The decoding solution applied to a certain device firmware may not be applicable to other firmware, which brings difficulties to large-scale and batch automation analysis of the firmware. There are three common tools for decompressing any firmware image: binwalk[2], FRAK[3], and Binary Analysis Toolkit (BAT)[4]. Craig Heffner developed a well-known firmware decompression tool, binwalk, which uses pattern matching to locate and slice files in binary firmware, and it also extracts metadata such as version licenses. Cui et al. first proposed an unpacking toolkit FRAK, which evaluates from the data provided by the equipment service provider and the FRAK-supported model, and the decompression success rate is relatively low. Tjaldur Software designed the Binary Analysis Toolkit (BAT). It used the name GPLtool, originally intended to violate the GPL's detection. It recursively extracted files from the binary firmware and used the known strings in the GPL project. In addition, BAT also supports document segmentation.

B. Binary file association

In the field of binary file association, the detection of code clones is mostly used. Hemel et al. proposed a binary analysis method [5] that extracts the ASCII string in a binary file to find out if the binary file has a known source code or binary and does not support Unicode strings. Hemel et al. also proposed a large-scale code-cloning detection method[6], which was completed by comparing the Linux version with vendor-specified

variables, but this scheme is only applicable when source code is available. In the field of firmware security analysis, Costin et al. proposed a method for finding similar files in different firmware through fuzzy hash classification[7], but the fuzzy hashing method needs to compare the fuzzy hash of each file in twos, and it cannot be large. Scale application.

In the actual firmware file association, we need to directly associate the binary files, and not rely on the source code. The features need to be able to adapt to the cross-optimization option file association across the instruction set architecture. At the same time, the association algorithm must be efficient enough to be able to perform large Scale association analysis needs to be completed using a new file association method.

C. Protocol association

There are many researches on protocol function comparison and clustering in the field of malware analysis [11]. However, the common idea of these methods is to first abstract the function into fingerprints that represent its semantic features, and then use the fingerprint to compare two functions and calculate the distance between two functions, finnaly cluster the functions based on the distance. These distance-based clustering algorithms require a time complexity of O(N2) when clustering a library of N functions. This will be unbearable for the complexity O(N2) of firmware library with N millions. In the field of similarity calculation of web pages and texts, the time complexity of the clustering method based on min hash (MinHash) proposed by Andrei Broder [12] is only O(N). However, the clustering algorithm based on the minimum hash is not suitable for the clustering of malware functions. This is because the malware authors usually do the fuzzification of the malware code, which will greatly reduce the clustering performance. However, the research team found that the clustering algorithm based on the minimum hash is very suitable for the clustering of firmware functions after investigation. This is because the device vendors rarely compile the firmware and blur the firmware, and the same series of products are different. The compiler optimization options used by the firmware version are often the same, which makes the functions in these firmware have a very high similarity. Using the minimum hash clustering algorithm can get good results by extracting the appropriate features.

In the actual implementation of a large-scale firmware function association, if the firmware function library is not clustered in advance, the associated overhead will be excessive. The experimental data in [13] shows that offline execution of 1140 executable files takes 57 CPU days. Usually firmware contains dozens to dozens of executable files, and this is only offline processing. The time does not include the time associated with the online connection. After preliminary investigation, the research team showed that if clustering is performed on the firmware library in advance, the number of effective functions in the firmware library can be effectively reduced to 20 percent of the total number of functions, which will greatly reduce the correlation of large-scale functions. Time overhead.

In this paper, a kind of industrial control device firmware protocol vulnerabilities associated technology, through the technology of information extraction from the mass grid device firmware device attributes and extract the industrial control system, the characteristics of the construction of industrial control system device firmware and published vulnerability information correlation, faster in the industrial control equipment safety inspection found vulnerabilities

III. MULTI-GRANULARITY FEATURE EXTRACTION

A. Extraction of firmware-level features

1) Vendor, product, version number and release date

At the same time as the firmware is crawled from internet web server, we will store the climbed firmware according to the manufacturer and product category. On the firmware release page, we will extract the firmware version number and release time and use it as a firmware level feature.

2) The file system

The file systems involved in the firmware mainly include yaffs, cramfs, squashfs, jffs2, and so on. In decoding the firmware, we use the binwalk tool to identify the file system type as a feature of the firmware.

3) Instruction Set Architecture Type

The main instruction set architecture is PowerPC, ARM, MIPS, etc. Since there may be multiple processors on an embedded device, the binary in the firmware may correspond to multiple instruction set architectures. We use the union of all binary file instruction set architectures in the firmware as the instruction set architecture of the firmware. For details on how to identify the instruction set architecture, see the firmware preprocessing section.

B. Extraction of file-level features

1) Character string characteristics

In order for binaries to work efficiently, the code in the firmware is barely confusing, which allows us to extract printable strings from the firmware. We use the Strings tool to collect the string generated by the code segment instruction sequence in the binary file and the symbol string of the standard library in the symbol table. For binary files with ELF headers, we can use the -d parameter to extract the strings in the data segment. For binary files without ELF headers, we use the --byte parameter to filter out strings of more than 6 characters to eliminate noise. The extracted string information can be used as a file feature.

2) Operation code characteristics

Opcode (Operation Code) is part of a machine language instruction. Different instructions are represented by different encodings of this field of the

opcode, and each type of code represents an instruction. N-grams are a method based on the Statistical Language Model (SLM) and have been widely used in the field of Natural Language Processing (NLP). For the executable file, after disassembling, N-grams can be used as a sliding window of length N, intercepting each N opcodes and collecting a series of overlapping sub-opcode sequences as features. The byte sequence in binary is equivalent to the letter or letter sequence in the text classification, and the opcode sequence is equivalent to the word or word sequence. The opcode sequence has more definite meaning and higher reliability.

C. Extraction of function-level features

1) Extraction of numerical and structural features

The numerical characteristics of functions refer to the features that can be numerically processed in functions, such as the number of instructions, the number of basic blocks, and the size of a stack. Difficulties and ideas for the extraction of functional numerical features are as follows:

The extracted features should ignore the differences brought about by different platforms. Even if the same source code disassembles and assembles the binary files compiled for different platforms, its instruction set, registers, addressing modes, and intra-function control flow diagrams are also quite different. This requires that the extracted features can ignore the differences brought about by different platforms. Bitstreams, instruction sets or register names are heavily dependent on the platform, so the selected features cannot be based on these objects. After the binary file is disassembled and information is extracted, most of the information is expressed as a function call graph, a series of assembly functions, and a series of function control flow graphs. Through the experimental observation, it is found that the same source code is compiled for different platforms, and the call relations between the functions are basically the same; some basic attributes of the function, such as the calling string, also have cross-platform features; as for the function control flow graph, different platforms may result in The details of the map change, but some descriptive features, such as size, do not change dramatically. Therefore, this paper starts from the three aspects of function call relations, function basic information and function control flow graph information, and extracts weaker platform dependent features.

Numerical processing of assembler programs and diagrams. Since the objects of analysis are assembler programs and graphs, it is not possible to enter the neural network directly. This makes it necessary to numerically process the assembler program and graph. The numerical features provided by assembler programs and diagrams are numerous, and the cross-platform nature and differentiation of each feature are also not the same. For assembler programs, you can extract the stack, string, and percentage of special instructions. For graphs, analysis can be performed from point information, degree information, and path information.

2) The specific extraction process

The function call graph is a directed graph. A node represents a function, and a directed edge represents a call relationship between functions. The function call graph is analyzed, the number of calls to the function by other functions callt and the number of calls to other functions of the function *callf*, as well as the number of calls to the *callt2* and *callf2* after deduplication, constitute the call relationship characteristics.

Analyze the basic function information, calculate the stack space *stack*, code size *code*, call string number *str*, and call string set *strSet*. Perform instruction analysis on the function, the number of statistical instructions inst, the number of jump instructions *jump*, jump instruction percentage *jump*, the use of formula (3-1) calculation instruction entropy instEpt and jump instruction entropy *jumpEpt*, the P_k indicate the the proportion of instruction k. The above features constitute the basic features of the function.

$$Entropy = -\sum_{k=1}^{n} p_k lb(p_k) \tag{3-1}$$

One function corresponds to a function control flow graph (CFG). Each node in the graph corresponds to a basic block in the function. The directed edges between nodes correspond to the jump relations between basic blocks. The function control flow graph is analyzed from three aspects: point, degree and path.

Perform edge analysis on the function control flow graph. Calculate the number of *nodes* and the number of *edges*; According to formula (3-2), calculate the *density* of the graph. The above constitutes the characteristics of the edge attributes of the CFG map.

$$density = \frac{2 \times edge}{node(node - 1)} \tag{3-2}$$

Perform path analysis on the function control flow graph, use the Floyd or Dijkstra algorithm to calculate the minimum distance from the basic block to any basic block, construct the ascending distance sequence pathList, calculate the average path length *avePath*, and the map diameter (ie, the longest path) diameter, Calculate the graph link efficiency *effect* according to the formula (3-3), above constitute the CFG graph path characteristics.

$$effect = \frac{edge - avePath}{edge} \tag{3-3}$$

The function control flow graph is analyzed to calculate the degree of out-degree and in-degree of each node. The CFG graph is converted into undirected graph, and the degree of each node of the undirected CFG graph is calculated to form the in-degree ascending sequence iList. Ascending sequence *oList*, undirected ascending

sequence uList. From the three degree sequences, three maximum degrees *imax*, *omax*, *umax*, and three average degrees *ieva*, *oeva*, *ueva* are respectively calculated; and the entropy *uEpt* of the undirected degree is calculated from the equation (4-1). The clustering coefficient *cluster* of the graph is calculated according to the formula (3-4) (3-5), where c denotes the number of subgraph edges of the undirected CFG graph formed by all the neighboring nodes of the node *k*, and d_k denotes the degree of the node *k*. The above constitutes a CFG graph feature.

$$C_{C_k} = \frac{2c}{d_k(d_k - 1)} \tag{3-4}$$

$$cluster = \sum_{k=1}^{n} \frac{C_{C_k}}{n} \tag{3-5}$$

The above total 31-dimensional features as a numerical feature of a function.

IV. Similarity Based Association

In order to reduce the computational cost of the traditional collaborative filtering recommendation in finding nearest neighbors, ie, K-NN search, Gaede proposed the concept of "near neighbors" based on the principle of approximate nearest neighbor query. Minhash is a probability based on the above concept. Clustering algorithm, which clusters together a set of users' common scores of projects with a certain probability.

For feature sets A and B, the similarity can be defined by the Jaccard coefficient as: If the proportion of the same entries of two sets to all related entries is larger, the possibility of including the same result is greater. That is, if the Jaccard coefficient of A and B is larger, it is more likely that they contain the same content at the same time.

Calculate the Jaccard distance of two sets, usually using the Merge algorithm. Although the complexity of the Merge algorithm O(m+n) is not high (where m and n represent the length of two sets respectively), when the length of the set is very large, The algorithm still encounters some problems. The simplest and most obvious method at this time is to reduce the length of the collection by sampling. How to reduce the error caused by sampling is the main problem to be solved now.

For this problem, Broder proposes the concept of Min-wise Independent Permutation:

Let h(x) be a permutation function on the set X such that the probability of the value of the permutation of any element x in X is equal, then h(x) is a minimal independent permutation. It also proves the following properties: The order is a series of minimal independent permutation functions acting on A and B, and the sets SA, SB are constructed:

$$SA = (\min\{h1(A)\},\ \min\{h2(A)\},\ \ldots,\ \min\{hk(A)\}) \tag{4-1}$$

$$SB = (\min\{h1(B)\},\ \min\{h2(B)\},\ \ldots,\ \min\{hk(B)\}) \tag{4-2}$$

Where k is the number of permutation functions, then

$$\text{resemble}(A,\ B) = \text{resemble}(SA,\ SB) \tag{4-3}$$

With this conclusion, it is possible to estimate the Jaccard distances of two sets through the sampling method or permutation, that is, the similarity between the two sets. Generally, the size of SA and SB after the permutation transformation is much smaller than the original set A and B.

V. Evaluation

The operating environment of the experimental environment is Ubuntu 14.04, and the third-party software and tools include Metasploit, Binwalk, Firmware Mod Kit, IDA pro, Sulley, and so on. Due to the high hardware requirements of tools and virtual machines, the hardware configuration includes Intel core i7-4790U, 16 GB of memory. All software and tools will work on this hardware platform.

The selection of test equipment has two principles:

1) The firmware of the target embedded device can be easily obtained, and the firmware can be obtained from inside the device or from the device manufacturer without paying a large price;

2) The target embedded device firmware is protected by measures such as confusion and encryption.

We crawled 2156 router device firmware on the OpenWRT website[14], including backfire_10.02, barrie_breaker14.07, and chaos_calmer15.05. The feature extraction method proposed in this paper is used to extract the characteristics of 2156 router firmware, and the protocol function association database is established.

VI. Conclusion

In this paper, a kind of industrial control device firmware protocol vulnerabilities associated technology, through the technology of information extraction from the mass grid device firmware device attributes and extract the industrial control system, the characteristics of the construction of industrial control system device firmware and published vulnerability information correlation, faster in the industrial control equipment safety inspection found vulnerabilities

Acknowledgment

Supported by the the Science and Technology project of State Grid Corporation of China (No. 52110417001B), the National Natural Science Foundation of China (U1766215, 61702506).

REFERENCES

[1] E-ISAC, SANS, "Analysis of the Cyber Attack on the Ukrainian Power Grid," March 18, 2016. Available: https://ics.sans.org/media/EISAC_SANS_Ukraine_DUC_5.pdf

[2] C. Heffner. binwalk – firmware analysis tool designed to assist in the analysis, extraction, and re-verse engineering of firmware images.

[3] A. Cui. Embedded Device Firmware Vulnerability Hunting with FRAK. DefCon 20, 2012.

[4] Tjaldur Software Governance Solutions. Binary Analysis Tool (BAT).

[5] A. Hemel, K. T. Kalleberg, R. Vermaas, and E. Dolstra, "Finding software license violations through binary code clone detection," in Proceedings of the 8th Working Conference on Mining Software Repositories, ser. MSR '11. New York, NY, USA: ACM, 2011, pp. 63–72. [Online]. Available: http://doi.acm.org/10.1145/1985441.1985453.

[6] A. Hemel and R. Koschke, "Reverse engineering variability in source code using clone detection: A case study for linux variants of consumer electronic devices," in Proceedings of the 2012 19th Working Conference on Reverse Engineering, ser. WCRE '12. Washington, DC, USA: IEEE Computer Society, 2012, pp. 357–366. [Online]. Available: http://dx.doi.org/10.1109/WCRE.2012.45

[7] A. Costin, J. Zaddach, A. Francillon, D. Balzarotti, and S. Antipolis, "A large-scale analysis of the security of embedded firmwares." in USENIX Security, 2014, pp. 95–110.

[8] Briones I, Gomez A. Graphs, entropy and grid computing: Automatic comparison of malware[C]. Virus bulletin conference. 2008: 1-12.

[9] Lakhotia A, Preda M D, Giacobazzi R. Fast location of similar code fragments using semantic 'juice'[C]. Proceedings of the 2nd ACM SIGPLAN Program Protection and Reverse Engineering Workshop. ACM, 2013.

[10] David Y, Yahav E. Tracelet-based code search in executables[C] ACM Sigplan Conference on Programming Language Design and Implementation. ACM, 2014:349-360.

[11] Pewny J, Schuster F, Bernhard L, et al. Leveraging semantic signatures for bug search in binary programs[C]. Proceedings of the 30th Annual Computer Security Applications Conference. ACM, 2014: 406-415.

[12] Broder, Andrei Z., et al. "Min-Wise Independent Permutations." *symposium on the theory of computing* 60.3 (2000): 630-659.

[13] Pewny J, Garmany B, Gawlik R, et al. Cross-Architecture Bug Search in Binary Executables[C]. Security and Privacy (SP), 2015 IEEE Symposium on. IEEE, 2015.

[14] http://archive.openwrt.org/